

LOGICAL STRUCTURES

Lecture 6. Semantics of predicate logic

Advanced Program in Computer Science

Spring, 2025

Contents

- ① Models
- ② Semantic entailment

Models

- For propositional logic, to evaluate the formula we compute the truth table based on a given valuation.

Models

- For propositional logic, to evaluate the formula we compute the truth table based on a given valuation. For example,

$$(p \vee \neg q) \rightarrow (q \rightarrow p)$$

Models

- For propositional logic, to evaluate the formula we compute the truth table based on a given valuation. For example,

$$(p \vee \neg q) \rightarrow (q \rightarrow p)$$

- How can we evaluate formulas in predicate logic? For instance,

$$\forall x \exists y ((P(x) \vee Q(x)) \rightarrow (Q(x) \rightarrow P(x)))$$

Models

- For propositional logic, to evaluate the formula we compute the truth table based on a given valuation. For example,

$$(p \vee \neg q) \rightarrow (q \rightarrow p)$$

- How can we evaluate formulas in predicate logic? For instance,

$$\forall x \exists y ((P(x) \vee Q(x)) \rightarrow (Q(x) \rightarrow P(x)))$$

- We need a set of predicate symbols, a set of function symbols and also a set of values $\Rightarrow$ concept of *model*.

Models

Definition 1.1

Let $\mathcal{F}$ be a set of function symbols and $\mathcal{P}$ a set of predicate symbols, each symbol with a fixed number of required arguments.

Models

Definition 1.1

Let $\mathcal{F}$ be a set of function symbols and $\mathcal{P}$ a set of predicate symbols, each symbol with a fixed number of required arguments. A *model* $\mathcal{M}$ of the pair $(\mathcal{F}, \mathcal{P})$ consists of the following set of data:

Models

Definition 1.1

Let $\mathcal{F}$ be a set of function symbols and $\mathcal{P}$ a set of predicate symbols, each symbol with a fixed number of required arguments. A *model* $\mathcal{M}$ of the pair $(\mathcal{F}, \mathcal{P})$ consists of the following set of data:

- 1 A non-empty set A , the universe of concrete values;

Models

Definition 1.1

Let $\mathcal{F}$ be a set of function symbols and $\mathcal{P}$ a set of predicate symbols, each symbol with a fixed number of required arguments. A *model* $\mathcal{M}$ of the pair $(\mathcal{F}, \mathcal{P})$ consists of the following set of data:

- ① A non-empty set A , the universe of concrete values;
- ② for each nullary function symbol $f \in \mathcal{F}$, a concrete element $f^{\mathcal{M}}$ of A

Models

Definition 1.1

Let $\mathcal{F}$ be a set of function symbols and $\mathcal{P}$ a set of predicate symbols, each symbol with a fixed number of required arguments. A *model* $\mathcal{M}$ of the pair $(\mathcal{F}, \mathcal{P})$ consists of the following set of data:

- 1 A non-empty set A , the universe of concrete values;
- 2 for each nullary function symbol $f \in \mathcal{F}$, a concrete element $f^{\mathcal{M}}$ of A
- 3 for each $f \in \mathcal{F}$ with arity $n > 0$, a concrete function $f^{\mathcal{M}} : A^n \rightarrow A$ from A^n , the set of n -tuples over A , to A ; and

Models

Definition 1.1

Let $\mathcal{F}$ be a set of function symbols and $\mathcal{P}$ a set of predicate symbols, each symbol with a fixed number of required arguments. A *model* $\mathcal{M}$ of the pair $(\mathcal{F}, \mathcal{P})$ consists of the following set of data:

- 1 A non-empty set A , the universe of concrete values;
- 2 for each nullary function symbol $f \in \mathcal{F}$, a concrete element $f^{\mathcal{M}}$ of A
- 3 for each $f \in \mathcal{F}$ with arity $n > 0$, a concrete function $f^{\mathcal{M}} : A^n \rightarrow A$ from A^n , the set of n -tuples over A , to A ; and
- 4 for each $P \in \mathcal{P}$ with arity $n > 0$, a subset $P^{\mathcal{M}} \subseteq A^n$ of n -tuples over A .

Models

Definition 1.1

Let $\mathcal{F}$ be a set of function symbols and $\mathcal{P}$ a set of predicate symbols, each symbol with a fixed number of required arguments. A *model* $\mathcal{M}$ of the pair $(\mathcal{F}, \mathcal{P})$ consists of the following set of data:

- ① A non-empty set A , the universe of concrete values;
- ② for each nullary function symbol $f \in \mathcal{F}$, a concrete element $f^{\mathcal{M}}$ of A
- ③ for each $f \in \mathcal{F}$ with arity $n > 0$, a concrete function $f^{\mathcal{M}} : A^n \rightarrow A$ from A^n , the set of n -tuples over A , to A ; and
- ④ for each $P \in \mathcal{P}$ with arity $n > 0$, a subset $P^{\mathcal{M}} \subseteq A^n$ of n -tuples over A .

Note: The symbols f and P are just that: **symbols**, whereas $f^{\mathcal{M}}$ and $P^{\mathcal{M}}$ denote a concrete **function (or element) and relation in a model** $\mathcal{M}$, respectively.

Example 1.2

- Let $\mathcal{F} = \{i\}$ and $\mathcal{P} = \{R, F\}$; where i is a constant, F a predicate symbol with one argument and R a predicate symbol with two arguments.

Example 1.2

- Let $\mathcal{F} = \{i\}$ and $\mathcal{P} = \{R, F\}$; where i is a constant, F a predicate symbol with one argument and R a predicate symbol with two arguments.
- A model $\mathcal{M}$ contains:
 - a set of concrete elements A - which may be a set of states of a computer program.

Example 1.2

- Let $\mathcal{F} = \{i\}$ and $\mathcal{P} = \{R, F\}$; where i is a constant, F a predicate symbol with one argument and R a predicate symbol with two arguments.
- A model $\mathcal{M}$ contains:
 - a set of concrete elements A - which may be a set of states of a computer program.
 - The interpretations $i^{\mathcal{M}}$, $R^{\mathcal{M}}$, and $F^{\mathcal{M}}$ may then be a designated initial state, a state transition relation, and a set of final (accepting) states, respectively.

Example 1.2

- Let $\mathcal{F} = \{i\}$ and $\mathcal{P} = \{R, F\}$; where i is a constant, F a predicate symbol with one argument and R a predicate symbol with two arguments.
- A model $\mathcal{M}$ contains:
 - a set of concrete elements A - which may be a set of states of a computer program.
 - The interpretations $i^{\mathcal{M}}$, $R^{\mathcal{M}}$, and $F^{\mathcal{M}}$ may then be a designated initial state, a state transition relation, and a set of final (accepting) states, respectively.

For example, let $A = \{a, b, c\}$, $i^{\mathcal{M}} = a$,
 $R^{\mathcal{M}} = \{(a, a), (a, b), (a, c), (b, c), (c, c)\}$, and $F^{\mathcal{M}} = \{b, c\}$.

Example 1.2

- Let $\mathcal{F} = \{i\}$ and $\mathcal{P} = \{R, F\}$; where i is a constant, F a predicate symbol with one argument and R a predicate symbol with two arguments.
- A model $\mathcal{M}$ contains:
 - a set of concrete elements A - which may be a set of states of a computer program.
 - The interpretations $i^{\mathcal{M}}$, $R^{\mathcal{M}}$, and $F^{\mathcal{M}}$ may then be a designated initial state, a state transition relation, and a set of final (accepting) states, respectively.

For example, let $A = \{a, b, c\}$, $i^{\mathcal{M}} = a$,
 $R^{\mathcal{M}} = \{(a, a), (a, b), (a, c), (b, c), (c, c)\}$, and $F^{\mathcal{M}} = \{b, c\}$.

We informally check some formulas of predicate logic for this model:

 The formula

$$\exists y R(i, y)$$

- The formula

$$\exists y R(i, y)$$

states that there is a transition from the initial state to some state, which is **true** in our model.

- The formula

$$\exists y R(i, y)$$

states that there is a transition from the initial state to some state, which is **true** in our model.

- The formula

$$\neg F(i)$$

- The formula

$$\exists y R(i, y)$$

states that there is a transition from the initial state to some state, which is **true** in our model.

- The formula

$$\neg F(i)$$

indicates that the initial state is not a final, accepting state, which aligns with our model.

- The formula

$$\exists y R(i, y)$$

states that there is a transition from the initial state to some state, which is **true** in our model.

- The formula

$$\neg F(i)$$

indicates that the initial state is not a final, accepting state, which aligns with our model.

- The formula

$$\forall x \forall y \forall z (R(x, y) \wedge R(x, z) \rightarrow y = z)$$

- The formula

$$\exists y R(i, y)$$

states that there is a transition from the initial state to some state, which is **true** in our model.

- The formula

$$\neg F(i)$$

indicates that the initial state is not a final, accepting state, which aligns with our model.

- The formula

$$\forall x \forall y \forall z (R(x, y) \wedge R(x, z) \rightarrow y = z)$$

asserts that the transition relation is deterministic, which is **false** in our model.

- 1 The formula

$$\exists y R(i, y)$$

states that there is a transition from the initial state to some state, which is **true** in our model.

- 2 The formula

$$\neg F(i)$$

indicates that the initial state is not a final, accepting state, which aligns with our model.

- 3 The formula

$$\forall x \forall y \forall z (R(x, y) \wedge R(x, z) \rightarrow y = z)$$

asserts that the transition relation is deterministic, which is **false** in our model.

- 4 The formula

$$\forall x \exists y R(x, y)$$

- 1 The formula

$$\exists y R(i, y)$$

states that there is a transition from the initial state to some state, which is **true** in our model.

- 2 The formula

$$\neg F(i)$$

indicates that the initial state is not a final, accepting state, which aligns with our model.

- 3 The formula

$$\forall x \forall y \forall z (R(x, y) \wedge R(x, z) \rightarrow y = z)$$

asserts that the transition relation is deterministic, which is **false** in our model.

- 4 The formula

$$\forall x \exists y R(x, y)$$

implies that there are no deadlock states, which holds **true** in our model.

Models

Example 1.3

- Let $\mathcal{F} = \{e, \cdot\}$ and $\mathcal{P} = \{\leq\}$; where e is a constant, $\cdot$ is a function of 2 argument and $\leq$ a predicate symbol with two arguments.

Models

Example 1.3

- Let $\mathcal{F} = \{e, \cdot\}$ and $\mathcal{P} = \{\leq\}$; where e is a constant, $\cdot$ is a function of 2 argument and $\leq$ a predicate symbol with two arguments. We write $\cdot$ and $\leq$ in infix notation as in $(t_1 \cdot t_2) \leq (t \cdot t)$.

Models

Example 1.3

- Let $\mathcal{F} = \{e, \cdot\}$ and $\mathcal{P} = \{\leq\}$; where e is a constant, $\cdot$ is a function of 2 argument and $\leq$ a predicate symbol with two arguments. We write $\cdot$ and $\leq$ in infix notation as in $(t_1 \cdot t_2) \leq (t \cdot t)$.
- Model $\mathcal{M}$ consists:
 - set A of all binary strings, finite words over the alphabet $\{0, 1\}$, including the empty string ϵ ;

Models

Example 1.3

- Let $\mathcal{F} = \{e, \cdot\}$ and $\mathcal{P} = \{\leq\}$; where e is a constant, $\cdot$ is a function of 2 argument and $\leq$ a predicate symbol with two arguments. We write $\cdot$ and $\leq$ in infix notation as in $(t_1 \cdot t_2) \leq (t \cdot t)$.
- Model $\mathcal{M}$ consists:
 - set A of all binary strings, finite words over the alphabet $\{0, 1\}$, including the empty string ϵ ;
 - the interpretation $e^{\mathcal{M}}$ of e is just the empty word ϵ

Models

Example 1.3

- Let $\mathcal{F} = \{e, \cdot\}$ and $\mathcal{P} = \{\leq\}$; where e is a constant, $\cdot$ is a function of 2 argument and $\leq$ a predicate symbol with two arguments. We write $\cdot$ and $\leq$ in infix notation as in $(t_1 \cdot t_2) \leq (t \cdot t)$.
- Model $\mathcal{M}$ consists:
 - set A of all binary strings, finite words over the alphabet $\{0, 1\}$, including the empty string ϵ ;
 - the interpretation $e^{\mathcal{M}}$ of e is just the empty word ϵ
 - interpretation of $\cdot^{\mathcal{M}}$ of $\cdot$ is the concatenation of words.

Models

Example 1.3

- Let $\mathcal{F} = \{e, \cdot\}$ and $\mathcal{P} = \{\leq\}$; where e is a constant, $\cdot$ is a function of 2 argument and $\leq$ a predicate symbol with two arguments. We write $\cdot$ and $\leq$ in infix notation as in $(t_1 \cdot t_2) \leq (t \cdot t)$.
- Model $\mathcal{M}$ consists:
 - set A of all binary strings, finite words over the alphabet $\{0, 1\}$, including the empty string ϵ ;
 - the interpretation $e^{\mathcal{M}}$ of e is just the empty word ϵ
 - interpretation of $\cdot^{\mathcal{M}}$ of $\cdot$ is the concatenation of words. For example, $0110 \cdot^{\mathcal{M}} 1110 = 01101110$.

Models

Example 1.3

- Let $\mathcal{F} = \{e, \cdot\}$ and $\mathcal{P} = \{\leq\}$; where e is a constant, $\cdot$ is a function of 2 argument and $\leq$ a predicate symbol with two arguments. We write $\cdot$ and $\leq$ in infix notation as in $(t_1 \cdot t_2) \leq (t \cdot t)$.
- Model $\mathcal{M}$ consists:
 - set A of all binary strings, finite words over the alphabet $\{0, 1\}$, including the empty string ϵ ;
 - the interpretation $e^{\mathcal{M}}$ of e is just the empty word ϵ
 - interpretation of $\cdot^{\mathcal{M}}$ of $\cdot$ is the concatenation of words. For example, $0110 \cdot^{\mathcal{M}} 1110 = 01101110$.
 - interpretation $\leq^{\mathcal{M}}$ is the set $\{(s_1, s_2) \mid s_1 \text{ is a prefix of } s_2\}$

Models

Example 1.3

- Let $\mathcal{F} = \{e, \cdot\}$ and $\mathcal{P} = \{\leq\}$; where e is a constant, $\cdot$ is a function of 2 argument and $\leq$ a predicate symbol with two arguments. We write $\cdot$ and $\leq$ in infix notation as in $(t_1 \cdot t_2) \leq (t \cdot t)$.
- Model $\mathcal{M}$ consists:
 - set A of all binary strings, finite words over the alphabet $\{0, 1\}$, including the empty string ϵ ;
 - the interpretation $e^{\mathcal{M}}$ of e is just the empty word ϵ
 - interpretation of $\cdot^{\mathcal{M}}$ of $\cdot$ is the concatenation of words. For example, $0110 \cdot^{\mathcal{M}} 1110 = 01101110$.
 - interpretation $\leq^{\mathcal{M}}$ is the set $\{(s_1, s_2) \mid s_1 \text{ is a prefix of } s_2\}$

Note: s_1 is a *prefix* of s_2 if there is a binary word s_3 such that $s_1 \cdot^{\mathcal{M}} s_3 = s_2$.

Informal model checks

In model $\mathcal{M}$ in previous example:

- 1 The formula

$$\forall x ((x \leq x \cdot e) \wedge (x \cdot e \leq x))$$

Informal model checks

In model $\mathcal{M}$ in previous example:

- 1 The formula

$$\forall x ((x \leq x \cdot e) \wedge (x \cdot e \leq x))$$

says that every word is a prefix of itself concatenated with the empty word and conversely. This clearly **holds**.

Informal model checks

In model $\mathcal{M}$ in previous example:

- 1 The formula

$$\forall x ((x \leq x \cdot e) \wedge (x \cdot e \leq x))$$

says that every word is a prefix of itself concatenated with the empty word and conversely. This clearly **holds**.

- 2 The formula

$$\exists y \forall x (y \leq x)$$

Informal model checks

In model $\mathcal{M}$ in previous example:

- 1 The formula

$$\forall x ((x \leq x \cdot e) \wedge (x \cdot e \leq x))$$

says that every word is a prefix of itself concatenated with the empty word and conversely. This clearly **holds**.

- 2 The formula

$$\exists y \forall x (y \leq x)$$

says that there exists a word s that is a prefix of every other word. This also **holds** and it is the empty word ϵ .

Informal model checks

In model $\mathcal{M}$ in previous example:

- 1 The formula

$$\forall x ((x \leq x \cdot e) \wedge (x \cdot e \leq x))$$

says that every word is a prefix of itself concatenated with the empty word and conversely. This clearly **holds**.

- 2 The formula

$$\exists y \forall x (y \leq x)$$

says that there exists a word s that is a prefix of every other word. This also **holds** and it is the empty word ϵ .

- 3 The formula

$$\forall x \exists y (y \leq x)$$

Informal model checks

In model $\mathcal{M}$ in previous example:

- 1 The formula

$$\forall x ((x \leq x \cdot e) \wedge (x \cdot e \leq x))$$

says that every word is a prefix of itself concatenated with the empty word and conversely. This clearly **holds**.

- 2 The formula

$$\exists y \forall x (y \leq x)$$

says that there exists a word s that is a prefix of every other word. This also **holds** and it is the empty word ϵ .

- 3 The formula

$$\forall x \exists y (y \leq x)$$

says that every word has a prefix.

Informal model checks

In model $\mathcal{M}$ in previous example:

- 1 The formula

$$\forall x \forall y \forall z ((x \leq y) \rightarrow (x \cdot z \leq y \cdot z))$$

Informal model checks

In model $\mathcal{M}$ in previous example:

- 1 The formula

$$\forall x \forall y \forall z ((x \leq y) \rightarrow (x \cdot z \leq y \cdot z))$$

says that whenever a word s_1 is a prefix of s_2 then s_1s has to be a prefix of s_2s for every word s .

Informal model checks

In model $\mathcal{M}$ in previous example:

- 1 The formula

$$\forall x \forall y \forall z ((x \leq y) \rightarrow (x \cdot z \leq y \cdot z))$$

says that whenever a word s_1 is a prefix of s_2 then s_1s has to be a prefix of s_2s for every word s . This is **not correct**, for instance $s_1 = 01, s_2 = 011$ and $s = 0$.

Informal model checks

In model $\mathcal{M}$ in previous example:

- ④ The formula

$$\forall x \forall y \forall z ((x \leq y) \rightarrow (x \cdot z \leq y \cdot z))$$

says that whenever a word s_1 is a prefix of s_2 then s_1s has to be a prefix of s_2s for every word s . This is **not correct**, for instance $s_1 = 01$, $s_2 = 011$ and $s = 0$.

- ⑤ The formula

$$\neg \exists x \forall y ((x \leq y) \rightarrow (y \leq x))$$

Informal model checks

In model $\mathcal{M}$ in previous example:

- ④ The formula

$$\forall x \forall y \forall z ((x \leq y) \rightarrow (x \cdot z \leq y \cdot z))$$

says that whenever a word s_1 is a prefix of s_2 then s_1s has to be a prefix of s_2s for every word s . This is **not correct**, for instance $s_1 = 01$, $s_2 = 011$ and $s = 0$.

- ⑤ The formula

$$\neg \exists x \forall y ((x \leq y) \rightarrow (y \leq x))$$

says that there is no word s such that whenever s is a prefix of some other word s_1 , it is the case that s_1 is a prefix of s as well.

Informal model checks

In model $\mathcal{M}$ in previous example:

- ④ The formula

$$\forall x \forall y \forall z ((x \leq y) \rightarrow (x \cdot z \leq y \cdot z))$$

says that whenever a word s_1 is a prefix of s_2 then s_1s has to be a prefix of s_2s for every word s . This is **not correct**, for instance $s_1 = 01$, $s_2 = 011$ and $s = 0$.

- ⑤ The formula

$$\neg \exists x \forall y ((x \leq y) \rightarrow (y \leq x))$$

says that there is no word s such that whenever s is a prefix of some other word s_1 , it is the case that s_1 is a prefix of s as well. This is **true** because for all s then s is a prefix of $s0$ but $s0$ can not be a prefix of s .

Look-up table

In order to determine the truth values for $\forall x \dots$ and $\exists x \dots$ we must be able to modify the interpretation of the variable x .

Look-up table

In order to determine the truth values for $\forall x \dots$ and $\exists x \dots$ we must be able to modify the interpretation of the variable x .

Definition 1.4

A *look-up table* or *environment* for a universe A of concrete values is a function $l : \text{var} \rightarrow A$ from the set of variables var to A .

Look-up table

In order to determine the truth values for $\forall x \dots$ and $\exists x \dots$ we must be able to modify the interpretation of the variable x .

Definition 1.4

A *look-up table* or *environment* for a universe A of concrete values is a function $l : \text{var} \rightarrow A$ from the set of variables var to A .

For such an l , we denote by $l[x \rightarrow a]$ the look-up table which maps x to a and any other variable y to $l(y)$.

Look-up table

In order to determine the truth values for $\forall x \dots$ and $\exists x \dots$ we must be able to modify the interpretation of the variable x .

Definition 1.4

A *look-up table* or *environment* for a universe A of concrete values is a function $l : \text{var} \rightarrow A$ from the set of variables var to A .

For such an l , we denote by $l[x \rightarrow a]$ the look-up table which maps x to a and any other variable y to $l(y)$.

$$l[x \mapsto a](y) = \begin{cases} a & \text{if } y = x \\ l(y) & \text{if } y \neq x \end{cases}$$

Satisfaction Relation

Definition 1.5

Given a model $\mathcal{M}$ for a pair $(\mathcal{F}, \mathcal{P})$ and given an environment l ,

Satisfaction Relation

Definition 1.5

Given a model $\mathcal{M}$ for a pair $(\mathcal{F}, \mathcal{P})$ and given an environment l , we define the *satisfaction relation* $\mathcal{M} \models_l \phi$ for each logical formula ϕ over the pair $(\mathcal{F}, \mathcal{P})$ and look-up table l by structural induction on ϕ .

Satisfaction Relation

Definition 1.5

Given a model $\mathcal{M}$ for a pair $(\mathcal{F}, \mathcal{P})$ and given an environment l , we define the *satisfaction relation* $\mathcal{M} \models_l \phi$ for each logical formula ϕ over the pair $(\mathcal{F}, \mathcal{P})$ and look-up table l by structural induction on ϕ . If $\mathcal{M} \models_l \phi$ holds, we say that ϕ computes **T** in the model $\mathcal{M}$ with respect to the environment l .

Satisfaction Relation

Definition 1.5

Given a model $\mathcal{M}$ for a pair $(\mathcal{F}, \mathcal{P})$ and given an environment l , we define the *satisfaction relation* $\mathcal{M} \models_l \phi$ for each logical formula ϕ over the pair $(\mathcal{F}, \mathcal{P})$ and look-up table l by structural induction on ϕ . If $\mathcal{M} \models_l \phi$ holds, we say that ϕ computes $\top$ in the model $\mathcal{M}$ with respect to the environment l .

- P: if ϕ is of the form $P(t_1, t_2, \dots, t_n)$, then we interpret the terms $t_1, t_2, \dots, t_n$ in our set A by replacing all variables with their values according to l .

Satisfaction Relation

Definition 1.5

Given a model $\mathcal{M}$ for a pair $(\mathcal{F}, \mathcal{P})$ and given an environment l , we define the *satisfaction relation* $\mathcal{M} \models_l \phi$ for each logical formula ϕ over the pair $(\mathcal{F}, \mathcal{P})$ and look-up table l by structural induction on ϕ . If $\mathcal{M} \models_l \phi$ holds, we say that ϕ computes T in the model $\mathcal{M}$ with respect to the environment l .

- P: if ϕ is of the form $P(t_1, t_2, \dots, t_n)$, then we interpret the terms $t_1, t_2, \dots, t_n$ in our set A by replacing all variables with their values according to l .

In this way we compute concrete values $a_1, a_2, \dots, a_n$ of A for each of these terms, where we interpret any function symbol $f \in \mathcal{F}$ by $f^{\mathcal{M}}$.

Satisfaction Relation

Definition 1.5

Given a model $\mathcal{M}$ for a pair $(\mathcal{F}, \mathcal{P})$ and given an environment l , we define the *satisfaction relation* $\mathcal{M} \models_l \phi$ for each logical formula ϕ over the pair $(\mathcal{F}, \mathcal{P})$ and look-up table l by structural induction on ϕ . If $\mathcal{M} \models_l \phi$ holds, we say that ϕ computes **T** in the model $\mathcal{M}$ with respect to the environment l .

- **P**: if ϕ is of the form $P(t_1, t_2, \dots, t_n)$, then we interpret the terms $t_1, t_2, \dots, t_n$ in our set A by replacing all variables with their values according to l .

In this way we compute concrete values $a_1, a_2, \dots, a_n$ of A for each of these terms, where we interpret any function symbol $f \in \mathcal{F}$ by $f^{\mathcal{M}}$.

Now $\mathcal{M} \models_l P(t_1, t_2, \dots, t_n)$ holds iff $(a_1, a_2, \dots, a_n)$ is in the set $P^{\mathcal{M}}$.

Satisfaction Relation

- $\forall x$: the relation $\mathcal{M} \models_l \forall x \psi$ holds iff $\mathcal{M} \models_{l[x \rightarrow a]} \psi$ holds for **all** $a \in A$.

Satisfaction Relation

- $\forall x$: the relation $\mathcal{M} \models_l \forall x \psi$ holds iff $\mathcal{M} \models_{l[x \rightarrow a]} \psi$ holds for **all** $a \in A$.
- $\exists x$: $\mathcal{M} \models_l \exists x \psi$ holds iff $\mathcal{M} \models_{l[x \rightarrow a]} \psi$ holds for **some** $a \in A$.

Satisfaction Relation

- $\forall x$: the relation $\mathcal{M} \models_l \forall x \psi$ holds iff $\mathcal{M} \models_{l[x \rightarrow a]} \psi$ holds for **all** $a \in A$.
- $\exists x$: $\mathcal{M} \models_l \exists x \psi$ holds iff $\mathcal{M} \models_{l[x \rightarrow a]} \psi$ holds for **some** $a \in A$.
- $\neg$: the relation $\mathcal{M} \models_l \neg \psi$ holds iff it is **not the case** that $\mathcal{M} \models_l \psi$ holds.

Satisfaction Relation

- $\forall x$: the relation $\mathcal{M} \models_l \forall x \psi$ holds iff $\mathcal{M} \models_{l[x \rightarrow a]} \psi$ holds for **all** $a \in A$.
- $\exists x$: $\mathcal{M} \models_l \exists x \psi$ holds iff $\mathcal{M} \models_{l[x \rightarrow a]} \psi$ holds for **some** $a \in A$.
- $\neg$: the relation $\mathcal{M} \models_l \neg \psi$ holds iff it is **not the case** that $\mathcal{M} \models_l \psi$ holds.
- $\vee$: $\mathcal{M} \models_l \psi_1 \vee \psi_2$ holds iff $\mathcal{M} \models_l \psi_1$ **or** $\mathcal{M} \models_l \psi_2$ holds.

Satisfaction Relation

- $\forall x$: the relation $\mathcal{M} \models_l \forall x \psi$ holds iff $\mathcal{M} \models_{l[x \rightarrow a]} \psi$ holds for **all** $a \in A$.
- $\exists x$: $\mathcal{M} \models_l \exists x \psi$ holds iff $\mathcal{M} \models_{l[x \rightarrow a]} \psi$ holds for **some** $a \in A$.
- $\neg$: the relation $\mathcal{M} \models_l \neg \psi$ holds iff it is **not the case** that $\mathcal{M} \models_l \psi$ holds.
- $\vee$: $\mathcal{M} \models_l \psi_1 \vee \psi_2$ holds iff $\mathcal{M} \models_l \psi_1$ **or** $\mathcal{M} \models_l \psi_2$ holds.
- $\wedge$: $\mathcal{M} \models_l \psi_1 \wedge \psi_2$ holds iff $\mathcal{M} \models_l \psi_1$ **and** $\mathcal{M} \models_l \psi_2$ holds.

Satisfaction Relation

- $\forall x$: the relation $\mathcal{M} \models_l \forall x \psi$ holds iff $\mathcal{M} \models_{l[x \rightarrow a]} \psi$ holds for **all** $a \in A$.
- $\exists x$: $\mathcal{M} \models_l \exists x \psi$ holds iff $\mathcal{M} \models_{l[x \rightarrow a]} \psi$ holds for **some** $a \in A$.
- $\neg$: the relation $\mathcal{M} \models_l \neg \psi$ holds iff it is **not the case** that $\mathcal{M} \models_l \psi$ holds.
- $\vee$: $\mathcal{M} \models_l \psi_1 \vee \psi_2$ holds iff $\mathcal{M} \models_l \psi_1$ **or** $\mathcal{M} \models_l \psi_2$ holds.
- $\wedge$: $\mathcal{M} \models_l \psi_1 \wedge \psi_2$ holds iff $\mathcal{M} \models_l \psi_1$ **and** $\mathcal{M} \models_l \psi_2$ holds.
- $\rightarrow$: $\mathcal{M} \models_l \psi_1 \rightarrow \psi_2$ holds iff $\mathcal{M} \models_l \psi_2$ holds **whenever** $\mathcal{M} \models_l \psi_1$ holds.

Satisfaction Relation

- $\forall x$: the relation $\mathcal{M} \models_l \forall x \psi$ holds iff $\mathcal{M} \models_{l[x \rightarrow a]} \psi$ holds for **all** $a \in A$.
- $\exists x$: $\mathcal{M} \models_l \exists x \psi$ holds iff $\mathcal{M} \models_{l[x \rightarrow a]} \psi$ holds for **some** $a \in A$.
- $\neg$: the relation $\mathcal{M} \models_l \neg \psi$ holds iff it is **not the case** that $\mathcal{M} \models_l \psi$ holds.
- $\vee$: $\mathcal{M} \models_l \psi_1 \vee \psi_2$ holds iff $\mathcal{M} \models_l \psi_1$ **or** $\mathcal{M} \models_l \psi_2$ holds.
- $\wedge$: $\mathcal{M} \models_l \psi_1 \wedge \psi_2$ holds iff $\mathcal{M} \models_l \psi_1$ **and** $\mathcal{M} \models_l \psi_2$ holds.
- $\rightarrow$: $\mathcal{M} \models_l \psi_1 \rightarrow \psi_2$ holds iff $\mathcal{M} \models_l \psi_2$ holds **whenever** $\mathcal{M} \models_l \psi_1$ holds.

We sometimes write $\mathcal{M} \not\models_l \phi$ to denote that $\mathcal{M} \models_l \phi$ does not hold.

Example

Example 1.6

Let $\mathcal{F} = \{\text{alma}\}$ and $\mathcal{P} = \{\text{loves}\}$ where alma is a constant and loves a binary predicate.

Example

Example 1.6

Let $\mathcal{F} = \{\text{alma}\}$ and $\mathcal{P} = \{\text{loves}\}$ where alma is a constant and loves a binary predicate.

The model $\mathcal{M}$ consists of universe set $A = \{a, b, c\}$, the constant function $\text{alma}^{\mathcal{M}} = a$ and the predicate $\text{loves}^{\mathcal{M}} = \{(a, a), (b, a), (c, a)\}$.

Example

Example 1.6

Let $\mathcal{F} = \{\text{alma}\}$ and $\mathcal{P} = \{\text{loves}\}$ where alma is a constant and loves a binary predicate.

The model $\mathcal{M}$ consists of universe set $A = \{a, b, c\}$, the constant function $\text{alma}^{\mathcal{M}} = a$ and the predicate $\text{loves}^{\mathcal{M}} = \{(a, a), (b, a), (c, a)\}$. Now we want to check whether the model $\mathcal{M}$ satisfies

None of Alma's lovers' lovers love her.

Example

Example 1.6

Let $\mathcal{F} = \{\text{alma}\}$ and $\mathcal{P} = \{\text{loves}\}$ where **alma** is a constant and **loves** a binary predicate.

The model $\mathcal{M}$ consists of universe set $A = \{a, b, c\}$, the constant function $\text{alma}^{\mathcal{M}} = a$ and the predicate $\text{loves}^{\mathcal{M}} = \{(a, a), (b, a), (c, a)\}$. Now we want to check whether the model $\mathcal{M}$ satisfies

None of Alma's lovers' lovers love her.

$$\forall x \forall y (\text{loves}(x, \text{alma}) \wedge \text{loves}(y, x) \rightarrow \neg \text{loves}(y, \text{alma}))$$

Example

Example 1.6

Let $\mathcal{F} = \{\text{alma}\}$ and $\mathcal{P} = \{\text{loves}\}$ where **alma** is a constant and **loves** a binary predicate.

The model $\mathcal{M}$ consists of universe set $A = \{a, b, c\}$, the constant function $\text{alma}^{\mathcal{M}} = a$ and the predicate $\text{loves}^{\mathcal{M}} = \{(a, a), (b, a), (c, a)\}$. Now we want to check whether the model $\mathcal{M}$ satisfies

None of Alma's lovers' lovers love her.

$$\forall x \forall y (\text{loves}(x, \text{alma}) \wedge \text{loves}(y, x) \rightarrow \neg \text{loves}(y, \text{alma}))$$

Counter-example: the formula does not hold

Example

Example 1.6

Let $\mathcal{F} = \{\text{alma}\}$ and $\mathcal{P} = \{\text{loves}\}$ where **alma** is a constant and **loves** a binary predicate.

The model $\mathcal{M}$ consists of universe set $A = \{a, b, c\}$, the constant function $\text{alma}^{\mathcal{M}} = a$ and the predicate $\text{loves}^{\mathcal{M}} = \{(a, a), (b, a), (c, a)\}$. Now we want to check whether the model $\mathcal{M}$ satisfies

None of Alma's lovers' lovers love her.

$$\forall x \forall y (\text{loves}(x, \text{alma}) \wedge \text{loves}(y, x) \rightarrow \neg \text{loves}(y, \text{alma}))$$

Counter-example: the formula does not hold because

- $\mathcal{M} \models_{I[x \rightarrow a, y \rightarrow b]} \text{loves}(x, \text{alma}) \wedge \text{loves}(y, x)$ holds

Example

Example 1.6

Let $\mathcal{F} = \{\text{alma}\}$ and $\mathcal{P} = \{\text{loves}\}$ where **alma** is a constant and **loves** a binary predicate.

The model $\mathcal{M}$ consists of universe set $A = \{a, b, c\}$, the constant function **alma** $^{\mathcal{M}} = a$ and the predicate **loves** $^{\mathcal{M}} = \{(a, a), (b, a), (c, a)\}$. Now we want to check whether the model $\mathcal{M}$ satisfies

None of Alma's lovers' lovers love her.

$$\forall x \forall y (\text{loves}(x, \text{alma}) \wedge \text{loves}(y, x) \rightarrow \neg \text{loves}(y, \text{alma}))$$

Counter-example: the formula does not hold because

- $\mathcal{M} \models_{I[x \rightarrow a, y \rightarrow b]} \text{loves}(x, \text{alma}) \wedge \text{loves}(y, x)$ holds
- $\mathcal{M} \models_{I[x \rightarrow a, y \rightarrow b]} \neg \text{loves}(y, \text{alma})$ does not hold

Example

Example 1.7

Let $\mathcal{F} = \{\text{alma}\}$ and $\mathcal{P} = \{\text{loves}\}$ where alma is a constant and loves a binary predicate.

The model $\mathcal{M}$ consists of universe set $A = \{a, b, c\}$, the constant function $\text{alma}^{\mathcal{M}} = a$ and the predicate $\text{loves}^{\mathcal{M}} = \{(b, a), (c, b)\}$.

Example

Example 1.7

Let $\mathcal{F} = \{\text{alma}\}$ and $\mathcal{P} = \{\text{loves}\}$ where alma is a constant and loves a binary predicate.

The model $\mathcal{M}$ consists of universe set $A = \{a, b, c\}$, the constant function $\text{alma}^{\mathcal{M}} = a$ and the predicate $\text{loves}^{\mathcal{M}} = \{(b, a), (c, b)\}$. Now we want to check whether the model $\mathcal{M}$ satisfies

None of Alma's lovers' lovers love her.

Example

Example 1.7

Let $\mathcal{F} = \{\text{alma}\}$ and $\mathcal{P} = \{\text{loves}\}$ where **alma** is a constant and **loves** a binary predicate.

The model $\mathcal{M}$ consists of universe set $A = \{a, b, c\}$, the constant function $\text{alma}^{\mathcal{M}} = a$ and the predicate $\text{loves}^{\mathcal{M}} = \{(b, a), (c, b)\}$. Now we want to check whether the model $\mathcal{M}$ satisfies

None of Alma's lovers' lovers love her.

$$\forall x \forall y (\text{loves}(x, \text{alma}) \wedge \text{loves}(y, x) \rightarrow \neg \text{loves}(y, \text{alma}))$$

Example

Example 1.7

Let $\mathcal{F} = \{\text{alma}\}$ and $\mathcal{P} = \{\text{loves}\}$ where alma is a constant and loves a binary predicate.

The model $\mathcal{M}$ consists of universe set $A = \{a, b, c\}$, the constant function $\text{alma}^{\mathcal{M}} = a$ and the predicate $\text{loves}^{\mathcal{M}} = \{(b, a), (c, b)\}$. Now we want to check whether the model $\mathcal{M}$ satisfies

None of Alma's lovers' lovers love her.

$$\forall x \forall y (\text{loves}(x, \text{alma}) \wedge \text{loves}(y, x) \rightarrow \neg \text{loves}(y, \text{alma}))$$

Proof: the formula holds because in this case Alma has only one lover

Example

Example 1.7

Let $\mathcal{F} = \{\text{alma}\}$ and $\mathcal{P} = \{\text{loves}\}$ where **alma** is a constant and **loves** a binary predicate.

The model $\mathcal{M}$ consists of universe set $A = \{a, b, c\}$, the constant function $\text{alma}^{\mathcal{M}} = a$ and the predicate $\text{loves}^{\mathcal{M}} = \{(b, a), (c, b)\}$. Now we want to check whether the model $\mathcal{M}$ satisfies

None of Alma's lovers' lovers love her.

$$\forall x \forall y (\text{loves}(x, \text{alma}) \wedge \text{loves}(y, x) \rightarrow \neg \text{loves}(y, \text{alma}))$$

Proof: the formula holds because in this case Alma has only one lover and

- $\mathcal{M} \models_{I[x \rightarrow b, y \rightarrow c]} \text{loves}(x, \text{alma}) \wedge \text{loves}(y, x)$ holds

Example

Example 1.7

Let $\mathcal{F} = \{\text{alma}\}$ and $\mathcal{P} = \{\text{loves}\}$ where **alma** is a constant and **loves** a binary predicate.

The model $\mathcal{M}$ consists of universe set $A = \{a, b, c\}$, the constant function $\text{alma}^{\mathcal{M}} = a$ and the predicate $\text{loves}^{\mathcal{M}} = \{(b, a), (c, b)\}$. Now we want to check whether the model $\mathcal{M}$ satisfies

None of Alma's lovers' lovers love her.

$$\forall x \forall y (\text{loves}(x, \text{alma}) \wedge \text{loves}(y, x) \rightarrow \neg \text{loves}(y, \text{alma}))$$

Proof: the formula holds because in this case Alma has only one lover and

- $\mathcal{M} \models_{I[x \rightarrow b, y \rightarrow c]} \text{loves}(x, \text{alma}) \wedge \text{loves}(y, x)$ holds
- $\mathcal{M} \models_{I[x \rightarrow b, y \rightarrow c]} \neg \text{loves}(y, \text{alma})$ holds

Exercises

Question 1. Consider the formula

$$\phi = \forall x \forall y Q(g(x, y), g(y, y), z),$$

where Q and g have arity 3 and 2, respectively. Find two models $\mathcal{M}$ and $\mathcal{M}'$ with respective environments l and l' such that $\mathcal{M} \models_l \phi$ but $\mathcal{M}' \not\models_{l'} \phi$.

Semantic Entailment

- In propositional logic, the semantic entailment $\phi_1, \phi_2, \dots, \phi_n \models \psi$ holds iff whenever all $\phi_1, \phi_2, \dots, \phi_n$ evaluate to **T**, the formula ψ evaluates to **T**
- How can we define such a notion for formula in **predicate logic**?

Semantic Entailment

Definition 2.1

Let Γ be a (possibly infinite) set of formulas in predicate logic and ψ a formula of predicate logic.

Semantic Entailment

Definition 2.1

Let Γ be a (possibly infinite) set of formulas in predicate logic and ψ a formula of predicate logic.

- 1 Semantic entailment $\Gamma \models \psi$ holds iff for all models $\mathcal{M}$ and look-up tables l , whenever $\mathcal{M} \models_l \phi$ holds for all $\phi \in \Gamma$, then $\mathcal{M} \models_l \psi$ holds as well.

Semantic Entailment

Definition 2.1

Let Γ be a (possibly infinite) set of formulas in predicate logic and ψ a formula of predicate logic.

- 1 Semantic entailment $\Gamma \models \psi$ holds iff for all models $\mathcal{M}$ and look-up tables l , whenever $\mathcal{M} \models_l \phi$ holds for all $\phi \in \Gamma$, then $\mathcal{M} \models_l \psi$ holds as well.
- 2 Formula ψ is satisfiable iff there is some model $\mathcal{M}$ and some environment l such that $\mathcal{M} \models_l \psi$ holds.

Semantic Entailment

Definition 2.1

Let Γ be a (possibly infinite) set of formulas in predicate logic and ψ a formula of predicate logic.

- 1 Semantic entailment $\Gamma \models \psi$ holds iff for all models $\mathcal{M}$ and look-up tables l , whenever $\mathcal{M} \models_l \phi$ holds for all $\phi \in \Gamma$, then $\mathcal{M} \models_l \psi$ holds as well.
- 2 Formula ψ is satisfiable iff there is some model $\mathcal{M}$ and some environment l such that $\mathcal{M} \models_l \psi$ holds.
- 3 Formula ψ is valid iff $\mathcal{M} \models_l \psi$ holds for all models $\mathcal{M}$ and environments l in which we can check ψ .

Semantic Entailment

Definition 2.1

Let Γ be a (possibly infinite) set of formulas in predicate logic and ψ a formula of predicate logic.

- 1 Semantic entailment $\Gamma \models \psi$ holds iff for all models $\mathcal{M}$ and look-up tables l , whenever $\mathcal{M} \models_l \phi$ holds for all $\phi \in \Gamma$, then $\mathcal{M} \models_l \psi$ holds as well.
- 2 Formula ψ is satisfiable iff there is some model $\mathcal{M}$ and some environment l such that $\mathcal{M} \models_l \psi$ holds.
- 3 Formula ψ is valid iff $\mathcal{M} \models_l \psi$ holds for all models $\mathcal{M}$ and environments l in which we can check ψ .
- 4 The set Γ is consistent or satisfiable iff there is a model $\mathcal{M}$ and a look-up table l such that $\mathcal{M} \models_l \phi$ holds for all $\phi \in \Gamma$.

Example 2.2

Verify the semantic entailment

$$\forall x(P(x) \rightarrow Q(x)) \models \forall xP(x) \rightarrow \forall xQ(x)$$

Example 2.2

Verify the semantic entailment

$$\forall x(P(x) \rightarrow Q(x)) \models \forall xP(x) \rightarrow \forall xQ(x)$$

Let $\mathcal{M}$ be a model with look-up table l .

Example 2.2

Verify the semantic entailment

$$\forall x(P(x) \rightarrow Q(x)) \models \forall xP(x) \rightarrow \forall xQ(x)$$

Let $\mathcal{M}$ be a model with look-up table l .

Assume that $\mathcal{M} \models_l \forall x(P(x) \rightarrow Q(x))$ holds,

Example 2.2

Verify the semantic entailment

$$\forall x(P(x) \rightarrow Q(x)) \models \forall xP(x) \rightarrow \forall xQ(x)$$

Let $\mathcal{M}$ be a model with look-up table l .

Assume that $\mathcal{M} \models_l \forall x(P(x) \rightarrow Q(x))$ holds,

we need to prove that $\mathcal{M} \models_l \forall xP(x) \rightarrow \forall xQ(x)$ holds.

Example 2.2

Verify the semantic entailment

$$\forall x(P(x) \rightarrow Q(x)) \models \forall xP(x) \rightarrow \forall xQ(x)$$

Let $\mathcal{M}$ be a model with look-up table l .

Assume that $\mathcal{M} \models_l \forall x(P(x) \rightarrow Q(x))$ holds,

we need to prove that $\mathcal{M} \models_l \forall xP(x) \rightarrow \forall xQ(x)$ holds.

- **Case 1:** there exists a such that $\mathcal{M} \models_{l[x \rightarrow a]} P(x)$ does not hold.

Example 2.2

Verify the semantic entailment

$$\forall x(P(x) \rightarrow Q(x)) \models \forall xP(x) \rightarrow \forall xQ(x)$$

Let $\mathcal{M}$ be a model with look-up table l .

Assume that $\mathcal{M} \models_l \forall x(P(x) \rightarrow Q(x))$ holds,

we need to prove that $\mathcal{M} \models_l \forall xP(x) \rightarrow \forall xQ(x)$ holds.

- **Case 1:** there exists a such that $\mathcal{M} \models_{l[x \rightarrow a]} P(x)$ does not hold.
But then in this case $\mathcal{M} \models_l \forall xP(x) \rightarrow \forall xQ(x)$ holds.

Example 2.2

Verify the semantic entailment

$$\forall x(P(x) \rightarrow Q(x)) \models \forall xP(x) \rightarrow \forall xQ(x)$$

Let $\mathcal{M}$ be a model with look-up table l .

Assume that $\mathcal{M} \models_l \forall x(P(x) \rightarrow Q(x))$ holds,

we need to prove that $\mathcal{M} \models_l \forall xP(x) \rightarrow \forall xQ(x)$ holds.

- **Case 1:** there exists a such that $\mathcal{M} \models_{l[x \rightarrow a]} P(x)$ does not hold.
But then in this case $\mathcal{M} \models_l \forall xP(x) \rightarrow \forall xQ(x)$ holds.
- **Case 2:** $\mathcal{M} \models_{l[x \rightarrow a]} P(x)$ holds for all a .

Example 2.2

Verify the semantic entailment

$$\forall x(P(x) \rightarrow Q(x)) \models \forall xP(x) \rightarrow \forall xQ(x)$$

Let $\mathcal{M}$ be a model with look-up table l .

Assume that $\mathcal{M} \models_l \forall x(P(x) \rightarrow Q(x))$ holds,

we need to prove that $\mathcal{M} \models_l \forall xP(x) \rightarrow \forall xQ(x)$ holds.

- **Case 1:** there exists a such that $\mathcal{M} \models_{l[x \rightarrow a]} P(x)$ does not hold.
But then in this case $\mathcal{M} \models_l \forall xP(x) \rightarrow \forall xQ(x)$ holds.
- **Case 2:** $\mathcal{M} \models_{l[x \rightarrow a]} P(x)$ holds for all a .
But since for each a , $\mathcal{M} \models_{l[x \rightarrow a]} P(x) \rightarrow Q(x)$ holds.

Example 2.2

Verify the semantic entailment

$$\forall x(P(x) \rightarrow Q(x)) \models \forall xP(x) \rightarrow \forall xQ(x)$$

Let $\mathcal{M}$ be a model with look-up table l .

Assume that $\mathcal{M} \models_l \forall x(P(x) \rightarrow Q(x))$ holds,

we need to prove that $\mathcal{M} \models_l \forall xP(x) \rightarrow \forall xQ(x)$ holds.

- **Case 1:** there exists a such that $\mathcal{M} \models_{l[x \rightarrow a]} P(x)$ does not hold.
But then in this case $\mathcal{M} \models_l \forall xP(x) \rightarrow \forall xQ(x)$ holds.
- **Case 2:** $\mathcal{M} \models_{l[x \rightarrow a]} P(x)$ holds for all a .
But since for each a , $\mathcal{M} \models_{l[x \rightarrow a]} P(x) \rightarrow Q(x)$ holds.
 $\mathcal{M} \models_{l[x \rightarrow a]} Q(x)$ holds for all a .

Example 2.2

Verify the semantic entailment

$$\forall x(P(x) \rightarrow Q(x)) \models \forall xP(x) \rightarrow \forall xQ(x)$$

Let $\mathcal{M}$ be a model with look-up table l .

Assume that $\mathcal{M} \models_l \forall x(P(x) \rightarrow Q(x))$ holds,

we need to prove that $\mathcal{M} \models_l \forall xP(x) \rightarrow \forall xQ(x)$ holds.

- **Case 1:** there exists a such that $\mathcal{M} \models_{l[x \rightarrow a]} P(x)$ does not hold.
But then in this case $\mathcal{M} \models_l \forall xP(x) \rightarrow \forall xQ(x)$ holds.
- **Case 2:** $\mathcal{M} \models_{l[x \rightarrow a]} P(x)$ holds for all a .
But since for each a , $\mathcal{M} \models_{l[x \rightarrow a]} P(x) \rightarrow Q(x)$ holds.
 $\mathcal{M} \models_{l[x \rightarrow a]} Q(x)$ holds for all a . That means $\mathcal{M} \models_l \forall xQ(x)$ holds.

Example 2.2

Verify the semantic entailment

$$\forall x(P(x) \rightarrow Q(x)) \models \forall xP(x) \rightarrow \forall xQ(x)$$

Let $\mathcal{M}$ be a model with look-up table l .

Assume that $\mathcal{M} \models_l \forall x(P(x) \rightarrow Q(x))$ holds,

we need to prove that $\mathcal{M} \models_l \forall xP(x) \rightarrow \forall xQ(x)$ holds.

- **Case 1:** there exists a such that $\mathcal{M} \models_{l[x \rightarrow a]} P(x)$ does not hold.
But then in this case $\mathcal{M} \models_l \forall xP(x) \rightarrow \forall xQ(x)$ holds.
- **Case 2:** $\mathcal{M} \models_{l[x \rightarrow a]} P(x)$ holds for all a .
But since for each a , $\mathcal{M} \models_{l[x \rightarrow a]} P(x) \rightarrow Q(x)$ holds.
 $\mathcal{M} \models_{l[x \rightarrow a]} Q(x)$ holds for all a . That means $\mathcal{M} \models_l \forall xQ(x)$ holds.
Hence $\mathcal{M} \models_l \forall xP(x) \rightarrow \forall xQ(x)$ holds.

Example 2.3

Verify the semantic entailment

$$\forall x P(x) \rightarrow \forall x Q(x) \models \forall x (P(x) \rightarrow Q(x))$$

Example 2.3

Verify the semantic entailment

$$\forall x P(x) \rightarrow \forall x Q(x) \models \forall x (P(x) \rightarrow Q(x))$$

Counter-example: Consider the model $\mathcal{M}'$ with $A' = \{a, b\}$, $P^{\mathcal{M}'} = \{a\}$ and $Q^{\mathcal{M}'} = \{b\}$.

Example 2.3

Verify the semantic entailment

$$\forall x P(x) \rightarrow \forall x Q(x) \models \forall x (P(x) \rightarrow Q(x))$$

Counter-example: Consider the model $\mathcal{M}'$ with $A' = \{a, b\}$, $P^{\mathcal{M}'} = \{a\}$ and $Q^{\mathcal{M}'} = \{b\}$.

- Since $\mathcal{M}' \models_{l[x \rightarrow b]} P(x)$ does not hold,

Example 2.3

Verify the semantic entailment

$$\forall x P(x) \rightarrow \forall x Q(x) \models \forall x (P(x) \rightarrow Q(x))$$

Counter-example: Consider the model $\mathcal{M}'$ with $A' = \{a, b\}$, $P^{\mathcal{M}'} = \{a\}$ and $Q^{\mathcal{M}'} = \{b\}$.

- Since $\mathcal{M}' \models_{l[x \rightarrow b]} P(x)$ does not hold, so $\mathcal{M}' \models_l \forall x P(x)$ does not hold.

Example 2.3

Verify the semantic entailment

$$\forall x P(x) \rightarrow \forall x Q(x) \models \forall x (P(x) \rightarrow Q(x))$$

Counter-example: Consider the model $\mathcal{M}'$ with $A' = \{a, b\}$, $P^{\mathcal{M}'} = \{a\}$ and $Q^{\mathcal{M}'} = \{b\}$.

- Since $\mathcal{M}' \models_{l[x \rightarrow b]} P(x)$ does not hold, so $\mathcal{M}' \models_l \forall x P(x)$ does not hold. And thus $\mathcal{M}' \models_l \forall x P(x) \rightarrow \forall x Q(x)$ holds.
- $\mathcal{M}' \models_{l[x \rightarrow a]} P(x) \rightarrow Q(x)$ does not hold

Example 2.3

Verify the semantic entailment

$$\forall x P(x) \rightarrow \forall x Q(x) \models \forall x (P(x) \rightarrow Q(x))$$

Counter-example: Consider the model $\mathcal{M}'$ with $A' = \{a, b\}$, $P^{\mathcal{M}'} = \{a\}$ and $Q^{\mathcal{M}'} = \{b\}$.

- Since $\mathcal{M}' \models_{l[x \rightarrow b]} P(x)$ does not hold, so $\mathcal{M}' \models_l \forall x P(x)$ does not hold. And thus $\mathcal{M}' \models_l \forall x P(x) \rightarrow \forall x Q(x)$ holds.
- $\mathcal{M}' \models_{l[x \rightarrow a]} P(x) \rightarrow Q(x)$ does not hold so $\mathcal{M}' \models_l \forall x (P(x) \rightarrow Q(x))$ does not hold.

Exercises

Question 2.

Let ϕ be the sentence $\forall x \forall y \exists z (R(x, y) \rightarrow R(y, z))$ where R is a predicate symbol of two arguments.

- Ⓐ Let $A = \{a, b, c, d\}$ and $R^{\mathcal{M}} = \{(b, c), (b, b), (b, a)\}$. Do we have $\mathcal{M} \models \phi$?
- Ⓑ Let $A' = \{a, b, c\}$ and $R^{\mathcal{M}'} = \{(b, c), (a, b), (c, b)\}$. Do we have $\mathcal{M}' \models \phi$?

Exercises

Question 2.

Let ϕ be the sentence $\forall x \forall y \exists z (R(x, y) \rightarrow R(y, z))$ where R is a predicate symbol of two arguments.

- Ⓐ Let $A = \{a, b, c, d\}$ and $R^{\mathcal{M}} = \{(b, c), (b, b), (b, a)\}$. Do we have $\mathcal{M} \models \phi$?
- Ⓑ Let $A' = \{a, b, c\}$ and $R^{\mathcal{M}'} = \{(b, c), (a, b), (c, b)\}$. Do we have $\mathcal{M}' \models \phi$?

Question 3. Verify the semantic entailment

$$\forall x P(x) \vee \forall x Q(x) \models \forall x (P(x) \vee Q(x))$$

Exercises

Question 4. Which of the following semantic entailments hold?

- Ⓐ $\neg\forall x P(x) \models \exists x \neg P(x)$
- Ⓑ $\forall x\exists y R(x, y) \models \exists y\forall x R(x, y)$
- Ⓒ $\exists y\forall x R(x, y) \models \forall x\exists y R(x, y)$